

ФАХОВИЙ КОЛЕДЖ ОДЕСЬКОГО НАЦІОНАЛЬНОГО УНІВЕРСИТЕТУ
ІМЕНІ І. І. МЕЧНИКОВА

(повне найменування вищого навчального закладу)

Циклова комісія прикладної математики та інформаційних технологій

Освітньо-професійний ступінь: Фаховий молодший бакалавр

Спеціальність 113 Прикладна математика

(шифр і назва)

ЗАТВЕРДЖУЮ

Голова циклової комісії

“ _____ ” _____ 2026 року

З А В Д А Н Н Я

на кваліфікаційну роботу здобувачу освіти

Тарасенко Софія Єгорівна

(прізвище, ім'я, по батькові)

1. Тема кваліфікаційної роботи Розробка системи розпізнавання обличчя

керівник кваліфікаційної роботи Зеленін Сергій Володимирович

(прізвище, ім'я, по батькові, науковий ступінь, вчене звання)

2. Строк подання здобувачем роботи _____

3. Вихідні дані роботи: Мова програмування Python, бібліотека OpenCV, face_recognition, dlib, Flask; середовище розробки Visual Studio Code; операційна система MacOS; хмарна платформа Digital Ocean; фронтенд - React, TypeScript.

4. Зміст розрахунково-пояснювальної записки (перелік питань, які потрібно розробити) _____

1. Аналіз технологій розпізнавання обличчя

2. Обґрунтування вибору технологій розробки

3.Проектування гібридної системи розпізнавання обличчя

4. Реалізація та тестування

5. Перелік графічного матеріалу (з точним зазначенням обов'язкових креслень)

Рисунок 1.1 — Фрагмент створеної матриці на основі отриманих значень

Рисунок 1.2 — Виявлення обличчя за допомогою каскадних детекторів

Рисунок 1.3 — Точки вирівнювання обличчя

Рисунок 1.4 — Приклад ключових точок на обличчі, які використовуються для вирівнювання

Рисунок 1.5 — Структури графа для розпізнавання осіб

Рисунок 1.6 — Приклад використання геометричного методу для ідентифікації

Рисунок 1.7 — Ознаки Наг у методі Віола-Джонса

Рисунок 1.8 — Застосування методу гістограм ідентифікації обличчя

Рисунок 1.9 — Текстурний аналіз за методом LBPН

Рисунок 1.10 — Візуалізація векторів ознак в багатовимірному просторі

Рисунок 2.1 — Результат ідентифікації за допомогою бібліотеки OpenCV

Рисунок 2.2 — Результати ідентифікації обличчя бібліотекою Dlib на основі методів Віола-Джонса

Рисунок 2.3 — Схематична реалізація модульної клієнт-серверної системи

Рисунок 3.1 — Загальна архітектура гібридної системи

Рисунок 3.2 — Потік обробки відеоданих

Рисунок 3.3 — Схема попередньої обробки зображень

Рисунок 3.4 — Схема реалізації сучасного підходу розпізнавання обличчя

Рисунок 3.5 — Реалізація класичного режиму за допомогою методу LBPН

Рисунок 3.6 — Механізм адаптивного перемикання режимів за станом батареї

Рисунок 3.7 — Модульна структура програмного забезпечення

Рисунок 3.8 — Файлова структура бази даних

Рисунок 3.9 — Схематична реалізація робочого інтерфейсу з розпізнавання обличчя

Рисунок 4.1 — Етапи адаптивного препроцесингу зображення

Рисунок 4.2 — Практичний результат застосування CLANE до отриманого відеопотоку

Рисунок 4.3 — Порівняння гістограм яскравості до та після CLANE на основі вхідних даних

Рисунок 4.4 — Структура роботи нейромережевого режиму

Рисунок 4.5 — Структура роботи нейромережевого режиму (детальна)

Рисунок 4.6 — Схема роботи резервного режиму

Рисунок 4.7 — Результат роботи резервного режиму

Рисунок 4.8 — Алгоритм автоматичного перемикання режимів

Таблиця 3.1 — Механізм адаптивного перемикання між режимами

Таблиця 4.1 — Порівняння енергоспоживання режимів роботи

Таблиця 4.2 — Матриця помилок при тестуванні нейромережевого режиму

Таблиця 4.3 — Порівняльний аналіз ефективності всіх режимів роботи

6. Консультанти розділів проєкту (роботи)

Розділ	Прізвище, ініціали та посада консультанта	Підпис, дата	
		завдання видав	завдання прийняв

7. Дата видачі завдання _____

КАЛЕНДАРНИЙ ПЛАН

№ з/п	Назви етапів кваліфікаційної роботи	Строк виконання етапів роботи	Примітка
1	Аналіз існуючих методів розпізнавання обличчя	26.02.26-07.03.26	
2	Обґрунтування вибору технологій та архітектури системи	08.03.26-20.03.26	

3	Проектування гібридної системи та математичної моделі	21.03.26-31.03.26	
4	Реалізація програмного забезпечення	01.04.26-27.04.26	
5	Тестування системи та аналіз результатів	19.04.26-27.04.26	
6	Робота з оформленням	28.04.26-10.05.26	

Здобувач _____ **Софія Тарасенко**
(ім'я, прізвище)

Керівник роботи _____ **Сергій Зеленін**
(ім'я, прізвище)

АНОТАЦІЯ

Кваліфікаційна робота присвячена розробці системи розпізнавання обличчя для роботи в умовах нестабільної мережі передачі даних та електропостачання.

Мета проєкту – створити локально автономну систему розпізнавання обличчя, яка автоматично перемикається між режимами роботи при перериванні мережевого з'єднання та відключенні електропостачання.

У ході роботи було проаналізовано предметну область методів комп'ютерного зору. Розглянуто класичні методи (алгоритми) комп'ютерного зору та сучасні нейромережеві підходи. На основі проведеного аналізу сформульовано функціональні та нефункціональні вимоги до програмного забезпечення. Реалізацію виконано на основі модульної архітектури: фронтенд розроблено з використанням React та TypeScript, бекенд — фреймворк Flask. Розробка здійснювалася в середовищі Visual Studio Code. Запропонований застосунок розгорнуто на хмарній платформі DigitalOcean. Проведено комплексне тестування системи в різних умовах освітлення та за різного рівня обчислювальних ресурсів.

Практична цінність роботи полягає в створенні гібридної системи розпізнавання обличчя, яка забезпечує високу ефективність та стабільність функціонування при низькому споживанні ресурсів. Розроблена система може бути використана для відеоспостереження, в системах контролю доступу, мобільних додатках безпеки.

ЗМІСТ

ПЕРЕЛІК УМОВНИХ СКОРОЧЕНЬ	6
ВСТУП.....	7
РОЗДІЛ 1 АНАЛІЗ ТЕХНОЛОГІЙ РОЗПІЗНАВАННЯ ОБЛИЧЧЯ	9
1.1 Основні принципи розпізнавання обличчя	9
1.2 Класичні методи комп'ютерного зору	13
1.3 Сучасні нейромережеві системи ідентифікації обличчя	16
1.4 Проблеми та обмеження методів розпізнавання обличчя	18
РОЗДІЛ 2. ОБГРУНТУВАННЯ ВИБОРУ ТЕХНОЛОГІЙ РОЗРОБКИ	20
2.1 Аналіз мов програмування для реалізації систем комп'ютерного зору.....	20
2.2 Аналіз бібліотек та фреймворків для розпізнавання обличчя	21
2.3 Обґрунтування вибору середовища розробки	24
2.4 Обґрунтування вибору архітектури системи.....	25
2.5 Математична модель гібридної системи.....	26
2.5.1 Модель вхідного зображення	26
2.5.2 Модель етапу препроцесингу	27
2.5.3 Модель формування вектору ознак	27
2.5.4 Модель прийняття рішень	28
2.5.5 Модель адаптивного перемикання.....	29
РОЗДІЛ 3. ПРОЕКТУВАННЯ ГІБРИДНОЇ СИСТЕМИ РОЗПІЗНАВАННЯ ОБЛИЧЧЯ.....	31
3.1 Постановка задачі розробки гібридної системи розпізнавання обличчя	31
3.2 Архітектура гібридної системи розпізнавання обличчя.....	32
3.3 Процес обробки вхідних файлів	33
3.4 Вимоги до гібридної системи розпізнавання обличчя.....	34
3.5 Алгоритми попередньої обробки зображень	34
3.6 Реалізація детекції обличчя.....	35
3.7 Реалізація нейромережевого розпізнавання	35
3.8 Реалізація класичного режиму розпізнавання	36
3.9 Реалізація механізму адаптивного перемикання.....	37
3.10 Модульна структура програми	38
3.11 Структура бази даних	39
3.12 Інтерфейс користувача.....	40
РОЗДІЛ 4. РЕАЛІЗАЦІЯ ТА ТЕСТУВАННЯ СИСТЕМИ	41
4.1 Реалізація адаптивного препроцесингу.....	41
4.2 Реалізація нейромережевого режиму	42

4.3 Реалізація резервного режиму.....	44
4.4 Реалізація механізму автоматичного перемикання між режимами.....	45
4.5 Тестування енергоефективності автономного режиму	46
4.6 Кількісний аналіз точності розпізнавання	46
4.7 Повний аналіз результатів.....	47
4.8 Напрямки подальшого розвитку системи.....	48
ВИСНОВКИ.....	50
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ	52
ДОДАТОК А	55

ПЕРЕЛІК УМОВНИХ СКОРОЧЕНЬ

CV (Computer Vision) – комп’ютерний зір.

CPU (Central Processing Unit) – центральний процесор.

OpenCv (Open Source Computer Vision) – бібліотека компютерного зору з відкритим кодом.

HOG (Histogram of Oriented Gradients) – гістограма орієнтовних градієнтів.

LBPH (Local Binary Patterns Histograms) – гістограми локальних бінарних шаблонів.

LBP (Local Binary Patterns) - локальні бінарні шаблони.

CLAHE (Contrast Limited Adaptive Histogram Equalization) – адаптивне вирівнювання гістограм з обмеження контрасту.

CNN (Convolution Neural Network) – згорткова нейронна мережа.

FPS (Frames Per Second) – кількість кадрів за секунду.

FN (False Negative) – хибнонегативний результат.

FP (False Positive) – хибнопозитивний результат.

HTTP (HyperText Transfer Protocol) – протокол передачі гіпертексту.

ResNet (Residual Neural Network) – залишкова нейронна мережа.

TN (True Negative) – істинно негативний результат.

TP (True Positive) – істинно позитивний результат.

RGB (Red Green Blue) – колірна модель.

EBGM (Elastic Bunch Graph Matching) - методи порівняння на графах.

ВСТУП

Сучасний розвиток інформаційних технологій впроваджує швидко застосування біометричних систем розпізнавання особи в усіх сферах людської діяльності. Розпізнавання обличчя є невід'ємною частиною життя, починаючи від систем контролю в навчальних закладах до автоматизованих обліків відвідуваності. Незважаючи на швидкий прогрес запроваджених алгоритмів, більший відсоток створених систем ідентифікації працює при стабільному енергопостачанні та за наявності потужних обчислювальних ресурсів.

Під час практичної експлуатації систем дані вимоги не завжди виконуються, що призводить до зниження ефективності та відмови розпізнавання обличчя на базі нейронних мереж. Застосування класичних алгоритмів розпізнавання забезпечує енергоефективність, але при цьому знижує результат ідентифікації.

Актуальність роботи полягає в необхідності створення адаптивних систем ідентифікації обличчя, які самостійно пристосовуватимуться до зміни зовнішніх умов. В умовах сучасного світу ця технологія вважається актуальною через нестабільне електропостачання.

Метою роботи є проєктування гібридної системи розпізнавання обличчя, що забезпечує стабільну роботу в умовах автономного живлення від акумулятора шляхом автоматичного перемикання між режимами функціонування системи під керуванням адаптивного контролера.

Завдання дослідження:

- провести порівняльний аналіз основних методів ідентифікації обличчя;
- обґрунтування вибору технологічного стеку для реалізації гібридної системи;
- реалізувати модель адаптивного препроцесинг методом CLANE ;
- спроектувати адаптивний контролер для роботи системи в реальному часі;

- провести тестування системи при різних зовнішніх умовах та визначити кількісну оцінку.

Об'єктом дослідження є процеси автоматизованої ідентифікації осіб засобами машинного навчання та комп'ютерного зору в умовах нестабільної мережі передачі даних та електропостачання.

Робота базується на створенні функціонуючого програмного забезпечення гібридної системи, яка здатна експлуатуватися в навчальних закладах та офісних приміщеннях в умовах нестабільних зовнішніх факторів.

РОЗДІЛ 1 АНАЛІЗ ТЕХНОЛОГІЙ РОЗПІЗНАВАННЯ ОБЛИЧЧЯ

За останні десятиліття технології автоматизованої ідентифікації особи за зображенням перейшли від вузькоспеціалізованої направленості до загальнодоступних технологій. Сфери використання біометричних систем ідентифікації починаються від розблокування смартфонів за допомогою (Face ID) до фіксації правопорушень на дорогах. Технологічно спільною основою всіх алгоритмів є методи комп'ютерного зору (Computer Vision, CV).

Всі підходи до розпізнавання обличчя, незалежно від використаного методу розпізнавання, будуть перетворювати вихідне зображення на числовий вектор, придатний для порівняння з еталоном бази даних.

1.1 Основні принципи розпізнавання обличчя

Система розпізнавання обличчя представляє собою багатоетапний алгоритм, який послідовно перетворює вхідні сигнали в набір для подальшої ідентифікації обличчя. Результат кожного попереднього етапу виступає вхідними даними для наступного. Помилка на будь-якому з етапів буде негативно впливати на кінцевий результат.

Технології біометричного розпізнавання обличчя почали розвиватися наприкінці XX – на початку XXI століття. Після початку впровадження глибоких нейронних мереж, технології за короткий проміжок часу стали найпопулярнішою сферою ідентифікації. Повний цикл обробки вхідних даних складається з:

- 1) препроцесингу;
- 2) детекції;
- 3) вирівнювання;
- 4) формування вектору індивідуальних ознак обличчя;
- 5) порівняння.

Система розпізнавання обличчя з технічної точки зору вважається програмним комплексом реалізації послідовних конвеєрів, які перетворюють вхідний кадр від простого потоку до числового рішення ідентифікації обличчя. Спираючись на архітектуру системи, застосовуються як класичні алгоритми комп'ютерного зору, так і методи глибокого навчання.

Препроцесинг. Після отримання вхідного зображення система нормалізує отриманий кадр для коректного виконання наступних етапів. Математично операція перетворення виконується за допомогою рівняння:

$$Q[x, y] = f(I[x, y]). \quad (1.1)$$

де $Q[x, y]$ – нормалізоване значення інтенсивності пікселя, після застосування перетворення f ;

f – функція перетворення;

$I[x, y]$ – яскравість пікселя з координатами (x, y) , який знаходиться в діапазоні $[0, 255]$, тобто, задається в 8-бітній моделі кодування.

Для виконання нормалізації формується матриця інтенсивності елементів. Приклад матриці наведено на рисунку (1.1).

0	192	64	255	64	192	0
255	64	192	0	192	64	255
0	192	64	255	64	192	0
255	64	192	0	192	64	255
0	192	64	255	64	192	0
255	64	192	0	192	64	255
0	192	64	255	64	192	0

Рисунок 1.1 Фрагмент створеної матриці на основі отриманих значень

Етап детекції. Нормалізувавши зображення, система локалізує обличчя на вхідному кадрі. Кожне обличчя в кадрі буде описуватися за допомогою координат прямокутника із застосуванням каскадних детекторів для перевірки кожної ділянки кадру на наявність обличчя, як показано на рисунку (1.2).



Рисунок 1.2 — Виявлення обличчя за допомогою каскадних детекторів

Вирівнювання обличчя (Face Alignment). Після виявлення обличчя на зображенні воно приводиться до стандартизованого вигляду, що забезпечує інваріантність при подальшій обробці кадру на наступних етапах. Геометричне вирівнювання обличчя реалізується за формулою (1.2) афінного перетворення, координат точок обличчя. Реалізуючи лінійне перетворення площини, зберігаючи паралельність прямих з операціями масштабування, повороту та зсуву.

$$x' = s \cdot R \cdot x + t \quad (1.2)$$

де x' — вихідні координати тачки на файлі;

x — вхідні координати точки;

s — коефіцієнти масштабування;

R — матриця повороту на кут θ ;

t — вектор зсуву по X та Y .

Параметри перетворення враховуються автоматично на основі знайдених системою координат точок на обличчі (очі, ніс, рот), як показано на рисунку (1.3), для визначення використовується бібліотека `dlib` [12, с. 1479].

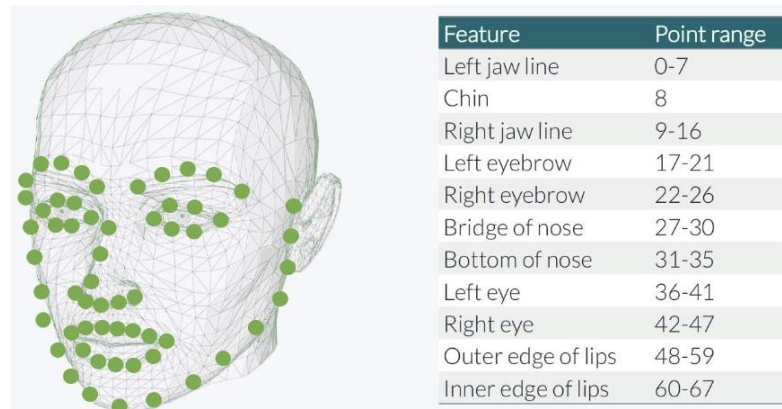


Рисунок 1.3 Точки вирівнювання обличчя

Формування вектора ознак. Отримане зображення зі стандартизованим положенням створює числовий вектор постійної розмірності, який описує біометричні ознаки обличчя. Залежно від архітектури моделі розмірність вектора може становити 128 або 512 [11, с. 815-823]. Кожна частина побудованого вектора містить інформацію про ключові частини обличчя кожної особи: просторові співвідношення, текстурні особливості.

Порівняння (Face Matching). Після формування вектора ознак система порівнює отримане значення з еталоном в базі даних. Математично це виконується за формулою (1.3) через обчислення косинусної міри схожості між векторами для визначення кута між ними в просторі ознак [11, с.819]:

$$\cos(\theta) = \frac{A * B}{||A|| * ||B||} \quad (1.3)$$

де A, B – вектори облич, результат визначається в діапазоні $[1, -1]$. Значення, наближене до 1 - дорівнює максимальній схожості.

Отримавши координати всіх ключових точок обличчя вони з'єднуються, як показано на рисунку (1.4).

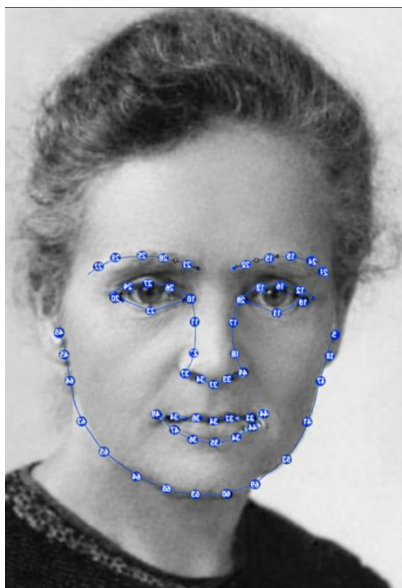


Рисунок 1.4 — Приклад ключових точок на обличчі, які використовуються для вирівнювання.

Кінцевий результат залежить від коректності виконання кожного з етапів. При поганій реалізації препроцесингу ідентифікація може коливатися при зміні умов освітлення або поганій якості зображення.

1.2 Класичні методи комп'ютерного зору

На початкових етапах впровадження систем детекції обличчя пошук виконувався за допомогою класичних алгоритмів розпізнавання осіб, на основі застосування комп'ютерного зору. Методи працюють на локальних автономних системах без потужного внутрішнього процесора. .

Метод порівняння на графіках (Elastic Bunch Matching). Найпростішим методом у виконанні вважається підхід, де кожне обличчя описується у вигляді графів, де вузлами виступають індивідуальні риси обличчя, а ребрами — відношення між ними, як показано на рисунку (1.5). Для виведення результату ідентифікації знаходяться схожі графи серед еталонних в базі даних.

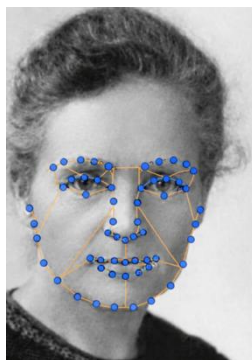


Рисунок 1.5 — Структури графа для розпізнавання осіб

Геометричні методи. Система вимірює відстань та кути між ключовими рисами обличчя (landmarks) , формуючи з них однаковий вектор, як показано на рисунку (1.6). Відстань обчислюється за евклідову формулою:

$$D_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} \quad (1.4)$$

Технології і досі вважаються актуальними через мінімальні витрати обчислювальних ресурсів без використання машинного навчання.

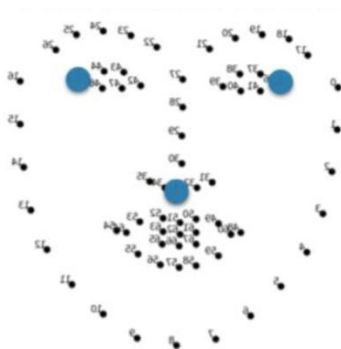


Рисунок 1.6 — Приклад використання геометричного методу для ідентифікації

Метод Віола-Джонса. Підхід, запропонований у 2001 році Paul Viola та Michael Jones. На момент створення вважався одним з перших методів ідентифікації обличчя, здатних працювати в режимі реального часу. Як зазначають Р. Viola та М. Jones [13, с. 511-518]. Така властивість методу

вважається провідною для роботи в Python. Враховується різниця інтенсивностей між описаними прямокутниками вхідного файлу, перетворюючи їх в ознаки Хаара, як показано на рисунку (1.7). Для прискорення обчислення різниці використовують формулу інтегрального зображення:

$$I(x, y) = \sum_{y' \leq y} I(x', y'), \quad \text{де } x' \leq x, y' \leq y \quad (1.5)$$

де $I(x, y)$ – значення інтегрального зображення в точці (x, y) ;

$I(x', y')$ – яс кравість пікселя вхідного файлу.

За формулою (1.5) обчислюється сума пікселів будь-якого прямокутника за 4 операції при константному часі $O(1)$.

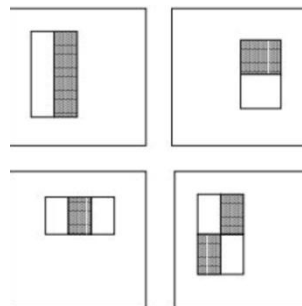


Рисунок 1.7 - Ознаки Наар у методі Віола Джонса

HOG (Histograms of Oriented Gradients). Метод аналізує градієнти зміни яскравості Кадр розбивається на прямокутники, як показано на рисунку (1.8), формуючи гістограму орієнтації градієнтів яскравості. Отримані ознаки передаються на класифікатор, створений на основі опорних векторів SVM (Support Vector Machine). Ділянки з обличчям обробляються одразу відкидаючи фонове зображення [6, с. 886-893].

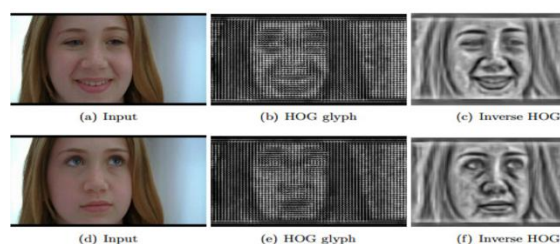


Рисунок 1.8 - Застосування методу гістограм ідентифікації обличчя

LBPН (Local Binary Patterns Histograms) – метод аналізує текстуру зображення шляхом порівняння яскравості пікселів [1, с. 2037-2041]. Для кожного пікселя з координатами $(x_c y_c)$ та інтенсивністю g_s пишеться бінарний код з p розташованими пікселями за формулою:

$$LBP(x_c y_c) = \sum_{p=0}^{P-1} (g_p - g_s) * 2^p \quad (1.6)$$

де g_p - інтенсивність p - го сусідній піксель;

s - порогова функція: $s(z) = 1$ при $z \geq 0$, інакше $s(z) = 0$.

Обчислення значення LBP накопичує гістограми і виступає ознаковим вектором для класифікації, як показано на рисунку (1.9).

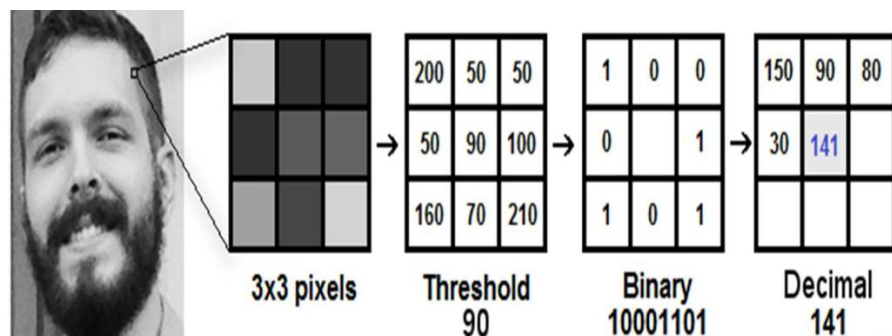


Рисунок 1.9 Текстурний аналіз за методом LBPН

1.3 Сучасні нейромережеві системи ідентифікації обличчя

Впровадження сучасних нейромережевих методів розпізнавання (Deep Learning) обличчя дозволило автоматизувати всі етапи перед ідентифікацією. Як зазначають Y. LeCun, Y. Bengio та G. Hinton [10, с. 436-444]. Використовується ручне проектування ознак, коли самостійно формуються ієрархічно подібне представлення даних.

В переважній більшості сучасних алгоритмів використовуються згорткові нейронні мережі (Convolutional Neural Networks, CNN) [9, с. 301-349]. Перші етапи реагують на локальні патерни – контур, зміна яскравості. При подальшій обробці формуються структурні елементи:

характерні риси обличчя та просторове співвідношення. При видачі результату кодуються абстрактні ознаки, які залишаються константними при зміні зовнішніх умов.

FaceNet був створений в 2015 році компанією Google. Реалізує відображення вхідного зображення x в точку евклідового простору за формулою фіксованого розміру:

$$f(x) \in R^{128} \text{ або } f(x) \in R^{512} \quad (1.7)$$

де x – вхідне зображення;

$f(x)$ – вектор ознак в багатовимірному просторі.

Візуалізація векторів ознак для знаходження обличчя наведено на рисунку (1.10).

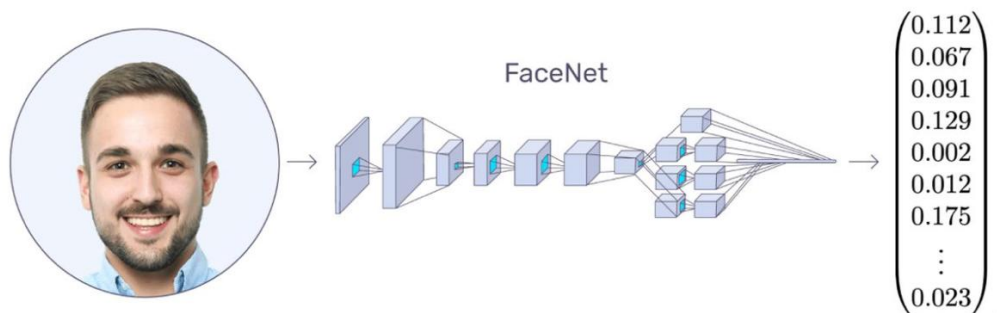


Рисунок 1.10 — Візуалізація векторів ознак в багатовимірному просторі

Навчання мережі FaceNet відбувається за допомогою функції втрат Triplet Loss на трійці базових зображень:

- *anchor* (a) – опорне зображення;
- *positive* (p) – інше зображення тієї самої людини;
- *negative* (n) – зображення іншої людини.

Triplet Loss наближує вектори *anchor* та *positive* та не приближатись до вектора *negative* за формулою:

$$L = \max(\|f(a) - f(p)\|^2 - \|f(a) - f(n)\|^2 + \alpha, 0) \quad (1.8)$$

де $\|f(a) - f(p)\|^2$ – квадратична відстань між відстанню однієї людини;

$\|f(a) - f(n)\|^2$ – квадратична відстань між відстанню різних людини;

α - мінімальна відстань між класами (0.2 або 0.5) [11, с. 815-823].

В 2019 році був створений метод Arc Face. Така технологія використовує функцію втрат з адаптивною кутовою гранню (Additive Angular Margin Loss) за формулою:

$$L = -\log \frac{e^{s(\cos\theta y + m)}}{e^{s(\cos\theta y + m)} + \sum_{j \neq y} e^{s \cos\theta j}} \quad (1.8)$$

де m — кутова грань, вона збільшує відстань між класами;

s — масштабний коефіцієнт;

θy — кут між векторами ознак.

1.4 Проблеми та обмеження методів розпізнавання обличчя

Існуючі методи ідентифікації обличчя на зображеннях та відеопотоках пройшли шлях від простих класичних алгоритмів до сучасних нейромережових моделей, дозволяючи працювати в умовах практичної експлуатації систем.

Класичні алгоритми Хаара, HOG та LBPН, не вимагають великого хмарного сховища в процесорі та не перевантажують його. При цьому результат ідентифікації буде напряму залежати від умов освітленості та якості вхідного відеопотоку. Неякісне освітлення або наявність тіней на кадрах призведе до зниження відсотка точності. У випадку, якщо особа поверне голову більше ніж на 30° від фронтального положення, ідентифікація виконується некоректно і видає різні результати. За наявності додаткових аксесуарів на обличчі: окулярів, головного убору, бороди, медичної маски — процес розпізнавання ускладнюється через локалізацію індивідуальних ознак особи на фотографії.

РОЗДІЛ 2. ОБГРУНТУВАННЯ ВИБОРУ ТЕХНОЛОГІЙ РОЗРОБКИ

Розробка системи ідентифікації обличчя базується на аналізі та обґрунтуванні вибору технологій внутрішнього стеку. При виборі враховувались можливості їх реалізації, продуктивності та здатності функціонування в умовах обмеженого зовнішнього середовища. Для формування критерії відбору технології враховувались:

- 1) наявність відповідних бібліотеки;
- 2) здатність працювати на автономних локальних системах;
- 3) швидкість виконання роботи;
- 4) коректна взаємодія з нейромережевими фреймворками.

2.1 Аналіз мов програмування для реалізації систем комп'ютерного зору

Для реалізації системи розпізнавання обличчя проведено порівняльний аналіз чотирьох найпоширеніших мов програмування.

Python — інтерпретована мова програмування, яка має простий синтаксис, скорочуючи написання коду. Мова містить в собі розвинену екосистему спеціалізованих бібліотек, яка здатна працювати з відеотрансляцією в реальному часі. Багаторівневі обчислення реалізуються на мові C++ автоматично підключаючи Python. Мова надає можливість створити автономну систему на звичайному комп'ютері без потреби у потужних графічних процесорах (GPU) та великих хмарних сховищах.

C++ — мова програмування з можливостями низькорівневої роботи на апаратних ресурсах. Використовується в складних задачах комп'ютерного зору. На основі C++ розроблені базові коди бібліотек OpenCV та dlib, які мають прив'язку до інших мов програмування[4, с. 3]. Мова застосовується в промислових системах реального часу, надаючи можливість розробнику ручне

управління пам'яттю та контроль над апаратними ресурсами, використовуючи нейронні мережі.

C# — мова, яка використовується для розробки веб-сервісів і є частиною системи .NET. Мова підходить для створення десктопних застосунків під операційну систему Windows. Містить в собі обмежену кількість бібліотек в галузі нейромережевого розпізнавання осіб.

Провідна мова в сфері серед веб-розробок — Javascript. Завдяки бібліотеці TensorFlow.js запускає нейромережі в браузері, а також надає доступ до бібліотек, написаних на C++. Однак, браузер накладає обмеження на доступ до внутрішніх ресурсів системи, де неможливо отримати дані про стан батареї чи навантаження на процесор CPU.

При розгляді мов програмування, для реалізації системи було обрано Python. Дана мова відповідає всім вимогам до створення гібридної системи розпізнавання обличчя, інтегруючи швидку роботу та можливості введення всіх необхідних бібліотек, підтримуючи роботу на власному ПК.

2.2 Аналіз бібліотек та фреймворків для розпізнавання обличчя

Для виконання детекції та ідентифікації обличчя до складу системи інтегруються бібліотеки та фреймворки. Кожен алгоритм описаний в системі відповідає за конкретний функціонал, використовуючи відповідні бібліотеки.

OpenCV (Open Source Computer Vision Library) — бібліотека з відкритим кодом. Написана мовою програмування C++, маючи прив'язку до Python, Java, MATLAB. Як зазначає S. Brahmabhatt [4, с. 3-5], бібліотека OpenCV охоплює понад 2500 алгоритмів комп'ютерного зору та забезпечує повний цикл обробки зображень. Результат ідентифікації осіб наведено на рисунку (2.1).

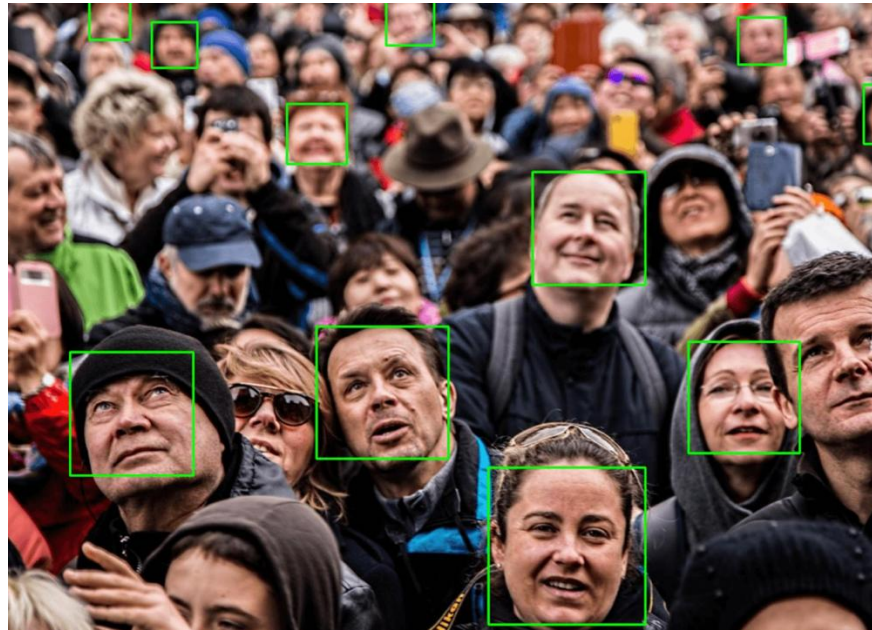


Рисунок 2.1 Результат ідентифікації за допомогою бібліотеки OpenCV

TensorFlow — фреймворк машинного навчання, розроблений компанією Google в 2015 році. Побудований за допомогою концепції обчислювання графів, де кожна математична операція представляється вузлом графа, а потік даних — ребрами. Виконується глибока оптимізація графів перед запуском, всі обчислення розподіляються між апаратними ресурсами — центральним (CPU) та графічним (GPU) процесорами.

PyTorch — фреймворк призначений для виконання задач з глибокого навчання, розроблений компанією Meta на основі проекту Torch від Facebook. Застосовуються динамічні обчислювальні графи, де архітектура мережі може змінюватися під час виконання.

Dlib — бібліотека, яка реалізує алгоритми машинного, написана мовою C++ з прив'язкою до Python. Головним елементом бібліотеки є детектор 68-ключових точок обличчя, які локалізують автоматичні орієнтири. Як зазначає D. G. Soni [12, с. 1479], усі отримані 68-ключових точок використовуються для геометричної нормалізації перед формування вектора ознак. Приклад застосування бібліотеки наведено на рисунку (2.2).

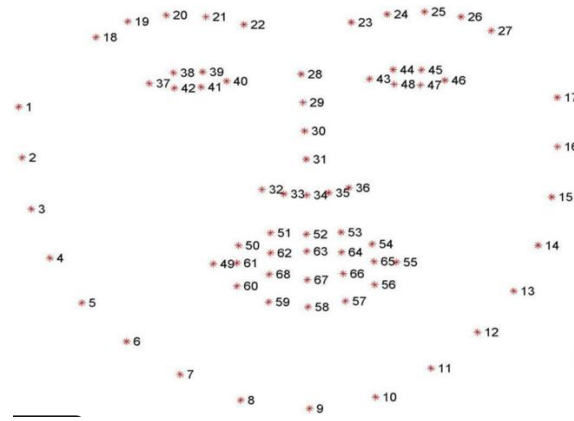


Рисунок 2.2 Результати ідентифікації обличчя бібліотекою Dlib на основі методів Віюла-Джонса

Face Recognition — Python-бібліотека для ідентифікації осіб, що реалізує повний алгоритм розпізнавання:

- детекція обличчя на вхідному кадрі;
- створення геометричної нормалізації;
- формування 128-вимірного вектора;
- порівняння отриманого вектора з пороговим в базі даних на основі відстані.

Deer Face — Python-бібліотека, яка об'єднує під єдиним програмним інтерфейсом декілька провідних моделей розпізнавання обличчя [7, с. 1701], Переключається між внутрішніми моделями та порівнює їх роботу при певному наборі даних. Окрім ідентифікації, бібліотека визначає додаткові атрибути особи, такі як вік, стать.

Flask — фреймворк для створення серверної частини систем. Працює за допомогою передання відеопотоків протоколом MPEG, обробляючи їх HTTP-запитами та на фінальному етапі створює REST API.

На основі описаних фреймворків та бібліотек є можливість створити технологічний стек в якому кожен аспект буде виконувати певну дію:

- OpenCv працює з відео потоком
- dlib та face_recognition реалізує метод нейромережевого розпізнавання
- Flask відповідає за сервісну частину програми

2.3 Обґрунтування вибору середовища розробки

Середовище розробки визначається сукупністю вимог, які будуть структурувати проект: повноцінна підтримка Python та всіх задіяних бібліотек, програмний доступ до параметрів контролю енергоспоживання та тестування обробки вхідних даних.

MacOS обрано як базова операційна система розробки, у зв'язку з тим, що створення та тестування системи планується на базі MacBook. Платформа будується на ядрі Unix при повній підтримці Python та всіх бібліотек без додаткових налаштувань. Використовується бібліотека `psutil` для доступу до інформації про заряд батареї та мережі живлення. Такі дані вважаються вихідними параметрами адаптивного контролера перемикання між режимами.

Для інтегрального середовища розробки (IDE) використано Visual Studio Code з розширенням Python. Вибір обґрунтований зручним керуванням віртуальним середовищем та покроковим тестуванням у процесі обробки вхідних зображень у реальному часі. Вбудований термінал в системі надає можливість запускати Flask-сервер та спостерігати за виводом в системі безпосередньо в процесі тестування.

Віртуальне середовище `venv` застосовувалося для ізоляції між проектами, не створюючи конфлікти. Бібліотеки `Dlib` та `FaceRecognition` містить в собі жорсткі вимоги до сумісних залежностей, тому розміщення повних пакетів всередині одного окремого віртуального середовища не призводить до конфліктів.

2.4 Обґрунтування вибору архітектури системи

Для реалізації системи обрано модульну клієнт-серверну архітектуру. Такий підхід дає можливість відокремити обробку вхідних кадрів та механізм моніторингу даних.

Клієнт-серверна модель була створена на базі Flask, реалізувавши систему як простий веб-додаток. Обраний фреймворк дозволяє виконувати неймережевий аналіз на вбудованому backendсервері, при цьому frontend відображає результат в браузері. Схематично створена модульна клієнт серверна архітектура показана на рисунку (2.3). Система працює через протокол HTTP, який дає можливість розпізнавати обличчя з будь-якого пристрою в локальній мережі.

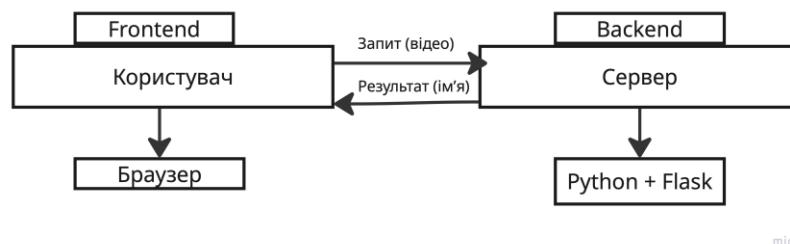


Рисунок 2.3 Схематично реалізація модульної клієнт-серверної системи

Ядром системи вважається логічний контролер, який дає можливість у реальному часі взаємодіяти з бібліотекою psutil. Передбачається 2 паралельні контури обробки, в яких контур високої точності базується на моделях згорткового неймережевого навчання. Використовуються при стабільному живленні для точної ідентифікації. Енергоефективний контур, який використовує каскад Хаара активується при зникненні енергетичних ресурсів та забезпечує функцію детекції при незначному навантаженні.

В роботі з даними система проектується, як множина взаємозамінних компонентів, в якій головним вважається незалежний модуль препроцесингу. Тобто, незалежний модуль виділяється в окремий функціональний блок, який дозволяє покращити якість вихідного файлу за допомогою методу CLANE,

незалежно від того, чи система працює на класичних методах, чи використовує нейромережевий підхід.

Даний алгоритм підкріплюється чіткою структурою набору даних (dataset). Архітектура спирається на файлову систему без використання заданих таблиць або складених баз даних. Що дозволяє автоматично прораховувати нових користувачів.

2.5 Математична модель гібридної системи

Для формального опису розробленої гібридної системи використовується математична модель. Вона описує всі етапи ідентифікації. Математично описуючи створену програму, кількісно оцінюється вплив кожного етапу на результат ідентифікації.

2.5.1 Модель. вхідного зображення

Система отримуючи вхідний файл переводить його в тривимірну матрицю значень за формулою:

$$I \in R^{H \times W \times C} \quad (2.1)$$

де H — висота зображення;

W — ширина;

C — кількість колірних каналів.

При форматі RGB значення $C = 3$, для зображень у відтінках сірого $C = 1$. Кожний елемент виведеної в матриці $I(x, y, c)$ відображає чисельне значення в діапазоні від 0 до 255. Всі кольорові кадри перетворюються на чорно-білі для зручності роботи за формулою:

$$I_{gray}(x, y) = 0.299 \cdot R(x, y) + 0.587 \cdot G(x, y) + 0.114 \cdot B(x, y) \quad (2.2)$$

2.5.2 Модель етапу препроцесингу

При нормалізації зображення буде розбиватися на прямокутні ознаки розміром $M \times M$ пікселів. Для кожної ознаки r будується гістограма яскравості $h_r(v)$, де $h \in v \in [0, 255]$ - рівень яскравості, а $h_r(v)$ — кількість пікселів з необхідним значенням.

Після отримання всіх гістограм будується розподіл яскравості за формулою :

$$CDF_r(v) = \frac{1}{M^2} \sum_{u=0}^v h_r(u) \quad , \quad (2.3)$$

де M^2 — загальна кількість пікселів в певному регіоні. Така функція дозволяє описати частку пікселів із яскравістю, яка не перевищує рівень v .

В даній системі використано метод нормалізації CLANE. Для того, щоб запобігти надмірному посиленню шумів, вводиться постійне відсікання β . Всі значення на гістограмах, які перевищують поставлений поріг- прибираються, а надлишок розподіляється між усіма рівнями за формулою:

$$h_{clapped}(v) = \begin{cases} \beta, & h_r > \beta \\ h_r(v) + \delta, & h_r(v) \leq \beta \end{cases} \quad , \quad (2.4)$$

де $\delta = \frac{\sum_{v: h_r > \beta} (h_r(v) - \beta)}{256}$ — розподілений надлишок.

Кінцеве перетворення для кожного пікселів ділянки r відбувається за формулою:

$$I'(x, y) = T_r(I_{gray}(x, y)) \quad , \quad (2.5)$$

де T_r — функція тонового відображення.

2.5.3 Модель формування вектору ознак

Після локалізації обличчя на вихідному кадрі системі необхідно представити обличчя як вектор ознак.

При активації нейромережевого режиму провідною є бібліотека dlib, яка локалізує 68 ключових точок на обличчі [13, с1479]. При отриманні всіх необхідних точок відбувається афінне перетворення для при зведення обличчя до стандартизованого вигляду за формулою:

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} \alpha \cos \theta & -\alpha \sin \theta \\ \alpha \sin \theta & \alpha \cos \theta \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} + \begin{pmatrix} t_x \\ t_y \end{pmatrix} \quad (2.6)$$

де α – коефіцієнт масштабування;

θ – кут повороту;

t_x та t_y – вектор зсуву.

При нормалізації зображення подається у 128-вимірний евклідовий простір ознак за формулою :

$$f = \varphi(I) \in \mathbb{R}^{128} \quad . \quad (2.7)$$

де I – нормалізоване зображення;

φ – *FaceNet*;

f – 128 – вимірний вектор ознак.

Мережа FaceNet навчена з використанням функції втрат TripletLoss, реалізуючи відображення φ . Зображення однієї особи утворюють компактну класифікацію в просторі $\mathbb{R}^{128}$, а зображення різних осіб — максимально віддалені точки одна від одної.

При активації класичного режиму вектор ознак формується методом LBPН. Для кожного пікселя обчислюється бінарний код через порівняння його інтенсивності g_c з інтенсивністю сусіднього пікселя P за формулою:

$$LBP(x_c y_c) = \sum_{p=0}^{P-1} s(g_p - g_c) \cdot 2^p \quad (2.8)$$

де s – порогова функція:

$$s(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

Всі обчислення значення LBP накопичуються в гістограмі h , яка буде виступати основою для подальшої класифікації.

2.5.4 Модель прийняття рішень

Коли система знаходиться на етапі визначення, формується вектор ознак текстового зображення, який порівнюється з пороговим значенням в базі даних.

При нейромережевому режимі використовується Евклідова відстань між текстовим вектором f_{test} та пороговим $f_{\text{ref},j}$ для кожної j – ї особи за формулою:

$$d_j = \sqrt{\sum_{m=1}^{128} (f_{\text{test},m} - f_{\text{ref},j})^2} \quad . \quad (2.9)$$

Система розпізнає особу з мінімальною відстанню та перевіряє відповідність з пороговим значенням за формулою:

$$y = \begin{cases} \text{особа } j^*, & \text{якщо } d_{j^*} = \min_j d_j \text{ та } d_{j^*} \leq \tau \\ \text{невідомий}, & \text{якщо } d_{j^*} > \tau \end{cases} \quad , \quad (2.10)$$

де $j^* = \arg$;

$\min_j d_j$ – індекс найближчої особи в базі;

θ – поріг класифікації;

τ – порогове значення.

В створеній гібридній системі порогове значення береться за 0.6.

При класичному режимі відбувається порівняння гістограм $h_{\text{ref},i}$ та $h_{\text{test},k}$ виконується як метрика χ^2 – квадрат за формулою:

$$d_j^{\chi^2} = \sum_{k=1}^K \frac{(h_{\text{test},k} - h_{\text{ref},j,k})^2}{h_{\text{ref},j,k} + \varepsilon} \quad , \quad (2.11)$$

де K – кількість бінів в гістограмі;

ε – мала константа для запобігання ділення на нуль при $h_{\text{ref},j,k} = 0$.

2.5.5 Модель адаптивного перемикавання

Застосувавши в системі метод адаптивного перемикавання, режими представляють собою функцію стану, яка залежить від 3 вимірних параметрів середовища за формулою:

$$\mathcal{M} = f(B, L_{\text{CPU}}, S) \quad , \quad (2.12)$$

де $B \in [0,100]$ – відповідає за рівень заряду батареї в відсотках;

$L_{\text{CPU}} \in [0,1]$ – відносне навантаження на процесор;

$S \in \{0,1\}$ – параметр стану підключення до батареї.

Логіка перемикання між режимами формально відображається через систему з пріоритетними значеннями від найкритичнішого до базового за формулою:

$$\mathcal{M} = \begin{cases} \text{Класичний} & S = 0, B \leq 25\%, \quad L_{CPU} \geq 0,80 \\ \text{Нейромережевий} & S = 1, B > 20\%, \quad L_{CPU} < 0,80 \end{cases} \quad (2.13)$$

Для того, щоб запобігти хибному перемиканню, застосовується зміна режиму лише тоді, коли нові умови зберігаються безперервно протягом певного часу за формулою:

$$\Delta \mathcal{M} \neq 0 \leftrightarrow \forall t' \in [t - \tau, t] : f(B(t'), L_{CPU}(t'), S(t')) = \mathcal{M}_{new} \quad (2.14)$$

Де $\tau = 10\text{с}$ – мінімальний стабільний .

РОЗДІЛ 3. ПРОЕКТУВАННЯ ГІБРИДНОЇ СИСТЕМИ РОЗПІЗНАВАННЯ ОБЛИЧЧЯ

Проектування програмної системи закладає архітектурні рішення, які визначають функціонал продуктивності програми для її експлуатації. Описується загальна структура розробленої гібридної системи ідентифікації обличчя та формуються загальні вимоги до функціонування

3.1 Постановка задачі розробки гібридної системи розпізнавання обличчя

Необхідно комплексно спроектувати та програмно реалізувати гібридну систему ідентифікації обличчя, яка буде працювати як при стабільному енергопостачанні та мережі, так і в умовах резервного режиму від акумулятора.

Процеси автоматизованої детекції та ідентифікації обличчя відбуваються за допомогою комп'ютерного зору та машинного навчання в умовах нестабільного живлення та зміни освітлення. Використані методи адаптивного керування алгоритмами розпізнавання обличчя повинні підвищити якість вхідних зображень під час роботи в реальному часі.

Розробляючи дану систему, необхідно інтегрувати модуль адаптивного препроцесингу. Метод CLANE вводиться для забезпечення нормалізації контрасту та яскравості відеопотоку незалежно від умов освітлення та якості файлів.

Проектується дворівнева архітектура обробки обличчя з реалізацією логічного контролера, який у реальному часі буде відстежувати параметр апаратного середовища, не перериваючи трансляцію.

Для реалізації серверної частини системи інтегровано з потоковою трансляцією відеопотоку через протокол та розробку інтерфейсу, який буде автоматично відображати результати ідентифікації та режим системи.

Створена система демонструє ефективність поєднання нейромережевого та класичного режимів. Ця адаптивна архітектура самостійно обирає оптимальний режим роботи залежно від поточного стану зовнішнього та внутрішнього середовища.

3.2 Архітектура гібридної системи розпізнавання обличчя

Проектування гібридної системи ідентифікації та детекції обличчя розробляється на основі адаптивної структури, стійкої до змін стану внутрішнього середовища роботи. Система архітектурно базується на клієнт-серверній моделі. Обрана модель вважається оптимальною основою фреймворку Flask. Такий підхід дозволяє системі розподіляє процеси входу даних, обробки та виводу точних результатів , як показано на рисунку (3.1).



Рисунок 3.1 Загальна архітектура гібридної системи

В системі з'єднані два підходи розпізнавання обличчя (AI та Haar), які працюють залежно від стану мережі та енергопостачання. При перемиканні система може видавати некоректні результати. Звертаючи увагу на дану проблему, було введено адаптивний контроль системи якості зображень, який дозволяє системі постійно слідкувати за станом живлення без впливу зовнішніх факторів. Такий підхід дозволяє створити залежність між умовами роботи та обробкою даних:

- якщо на пристрої стабільне мережеве з'єднання використовується нейромережева модель (AI);

- якщо присутні проблеми з енергопостачанням, система буде працювати від батареї на класичних алгоритмах (Haar).

3.3 Процес обробки вхідних файлів

Розроблена гібридна система ідентифікації обличчя працює у вигляді нескінченного циклу обробки зображень та відеопотоків. Кожен наступний етап впливає з попереднього, а вихідною точкою вважається захоплення даних з вхідного файлу, де система отримає послідовність кадрів. Отримана послідовність буде проходити повний цикл обробки зображень для детекції обличчя спираючись на дані з попередніх етапів. Схематичне зображення обробки потоку відеоданих представлено на рисунку (3.2).

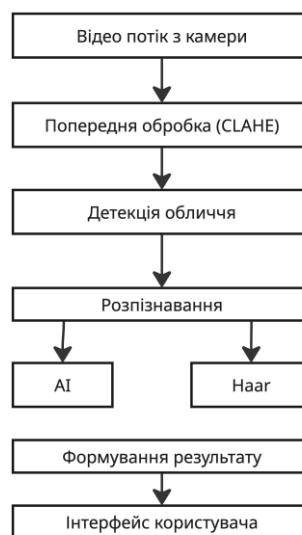


Рисунок 3.2 Потік обробки відеоданих

При отриманні вхідного потоку він перетворюється на числову матрицю інтенсивності пікселів для математичної обробки. У випадку надходження кольорового типу зображення буде використовуватися тривимірна матриця, де один шар зображення відповідає каналу RGB.

Після повної адаптивної обробки вхідного файлу дані перенаправляються в аналітичний блок. Система визначає метод аналізу в залежності від умов роботи системи. При стабільному живленні зображення

буде оброблятися сучасними методами, в яких пошук схожості відбувається в 128-вимірному просторі ознак. Коли система переходить на живлення від батареї, будуть задіяні каскадні класифікатори Хаара.

Після визначення методів, необхідних на відповідних етапах, виконується візуалізація результатів:

- на вхідному відео потоці відображається прямокутник навколо виявлених ключових рис обличчя;
- підписуються текстові дані з ідентифікацією людей (імена, прізвища);
- індикатор поточного режиму роботи;

Повністю ідентифіковане зображення відображається через протокол HTTP до браузера користувача в форматі потокового відео MPEG.

3.4 Вимоги до гібридної системи розпізнавання обличчя

Першочергово при створенні системи необхідною умовою було сформулювати чіткий перелік функціональних вимог програми. Такий підхід дозволяє працювати системі в різних умовах експлуатації.

Необхідною вимогою вважається робота системи в реальному часі. На цьому етапі система повинна відображати та одночасно обробляти вхідний відеопотік без затримок для користувача. Для цього в практичній частині системи забезпечується необхідна частота кадрів.

Умова автоматичного перемикавання режимів між нейромережевими та класичними відбувається без зупинки відеотрансляції. При використанні такої технології не потрібно перезавантажувати сервіс, зберігаючи попередні результати ідентифікації.

3.5 Алгоритми попередньої обробки зображень

Отримавши необхідну кількість вхідних файлів та збережені їх у базі даних, система починає обробку зображень (препроцесинг). Схематичне зображення адаптивного препроцесингу показано на рисунку (3.3).



Рисунок 3.3 Схема попередньої обробки

Для усунення недоліків детекції обличчя в приміщеннях з поганим освітленням інтегровано метод CLAHE (Contrast Limited Adaptive Histogram Equalization). Метод адаптивного вирівнювання гістограм на ділянках обличчя з обмеженою кількістю контрастності, розбиваючи вхідне зображення на прямокутники, працюючи з ними локально.

3.6 Реалізація детекції обличчя

Детекція обличчя в розробленій системі вважається обов'язковим етапом перед ідентифікацією. Програмі необхідно знайти обличчя на зображенні, визначити його геометричні координати та розписати область для аналізу.

При активації класичної моделі розпізнавання в умовах обмеженого енергопостачання використовуються Haar Cascades. Алгоритм описує різницю інтенсивності між сусідніми прямокутними ділянками кадру, які представляють собою ознаки Хаара. Класифікатор використовує загальноприйняті закономірності – очі темніші за ніс, підборіддя світліше за вилиці тощо.

3.7 Реалізація нейромережевого розпізнавання

Активація сучасних нейромережевих методів ідентифікації обличчя відбувається при стабільному з'єднанні системи з мережею. Реалізація

побудована за допомогою бібліотеки `face_recognition` на базі згорткової нейронної мережі ResNet з бібліотеки `dlib`. Схематично реалізацію виконання нейромережевого розпізнавання зображено на рисунку (3.4).

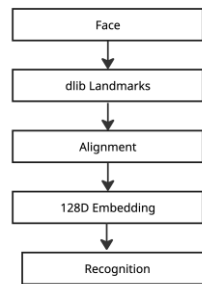


Рисунок 3.4 Схеми реалізації сучасного підходу реалізації обличчя

Процес нейромережевого розпізнавання проводиться на основі трьох аспектів:

1. Детекція – локалізація координат в прямокутнику;
2. Вирівнювання – проводиться за допомогою бібліотеки `dlib`, де визначаються 68-ключових точок обличчя та складається геометричне перетворення. Отримавши нормалізоване зображення система перетворює його на 128-вимірний вектор ознак;
3. Ідентифікація – порівнюються отримані вектори з пороговими значеннями векторів в базі даних через евклідову відстань. Якщо відстань менша за встановлений поріг $\theta = 0.6$ – система видає ім'я особи.

3.8 Реалізація класичного режиму розпізнавання

В умовах обмеженого енергопостачання система перемикається на класичні моделі розпізнавання. Реалізація режиму відбувається засобами бібліотеки `OpenCV` та поєднує локальні гістограми для ідентифікації.

Метод LBPН працює з текстурою зображення. Обличчя на зображенні переводиться в чорно-білий формат, нормалізується яскравість та приводиться до стандартного розміру 128×128 пікселів, як показано на рисунку (3.5).



Рисунок 3.5 Реалізація класичного режиму за допомогою методу LBPН

Яскравість кожного пікселя порівнюється з його восьми сусідами. З отриманої множини значень будується бінарний код для конкретного пікселя: якщо сусід світліший — 1, якщо темніший — 0. Після написання бінарних кодів для всіх пікселів вони представляються у вигляді гістограм. Кінцева ідентифікація обличчя виконується шляхом порівняння отриманих гістограм з пороговим значенням в базі даних.

3.9 Реалізація механізму адаптивного перемикання

В системі вбудовано автоматичний механізм перемикання між режимами. Стан апаратного живлення відстежується в реальному часі і перемикається за умов енергопостачання без участі користувача. Схематична реалізація механізму перемикання режимів за станом батареї та живлення представлено на рисунку (3.6).

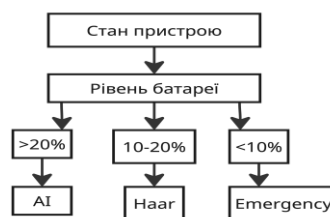


Рисунок 3.6 Механізм адаптивного перемикання режимів за станом батареї

Для реалізації механізму використовувалась бібліотека psutil. За допомогою обраної бібліотеки система контролює: рівень заряду батареї, підключення до мережі та навантаження на процесор.

Процес перемикання на нейромережевий режим відбувається, коли заряд батареї перевищує 20% при навантаженні процесора менше 80 FPS.

Класична модель системи буде активуватися автоматично при відключенні від мережі або заряді менше 25%.

Якщо заряд батареї нижче 10% вмикається резервний режим. Коли система виконує детекцію обличчя без ідентифікації, не перевантажуючи процесор.

Для захисту від самовільного перемикання було впроваджено умову, що зміна режимів відбувається лише тоді, коли умови зберігаються протягом 10–12 секунд.

Механізм адаптивного перемикання між класичними моделями та нейромережевими залежно від стану батареї показано в таблиці (3.1).

Таблиця 3.1 Механізм адаптивного перемикання між режимами.

Умова	Режим
Батарея >20%	Нейромережевий
Батарея <25%	Класичний
Батарея <10%	Резервний

3.10 Модульна структура програми

Програмне забезпечення в системі працює за принципом, де кожен функціональний блок відповідає за певну задачу і відображається через інтерфейс.

Коренева структура face_diploma/ містить у собі запуск сервера, модуль роботи препроцесингу, детекцію, ідентифікацію, адаптивне перемикання, візуалізацію інтерфейсу та директорію з базою даних, як показано на рисунку (3.7).

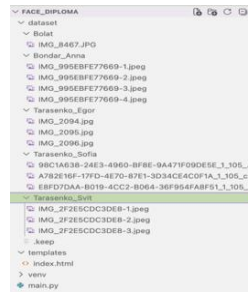


Рисунок 3.7 Модульна структура програмного забезпечення

Кожен етап в системі представляється окремим файлом в Python.

3.11 Структура бази даних

Створена гібридна система ідентифікації осіб не містить в собі реляційних баз даних; було використано файлову структуру, як показано на рисунку (3.8). Файлова система не потребує використання інших сервісів для роботи з базами даних.

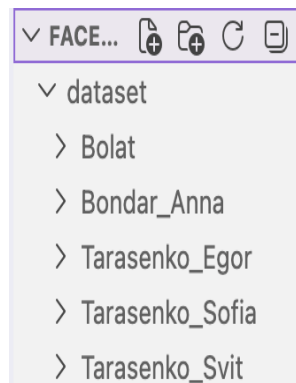


Рисунок 3.8 Файлова структура бази даних.

В базі даних зберігаються директорії, де ім'я кожної з них відповідає імені та прізвищу зареєстрованої особи. Зображення всередині папки зберігаються в форматах JPG або PNG. При запуску системи скануються всі директорії і для кожного зображення будується 128-вимірний вектор. Такі вектори зберігаються в оперативній пам'яті та використовуються під час порівняння з вхідним відеопотоком.

3.12 Інтерфейс користувача

Інтерфейс в системі реалізовується як окремий веб-додаток. Користувач має доступ через браузер до локальних мереж. Фронтенд-система побудована на HTML5 та JavaScript.

Для відображення обробленого відеопотоку з показаними результатами ідентифікації відображається в вікні відеоспостереження. В верхній частині вікна розташовується інформаційна панель, яка містить інформацію про список зареєстрованих осіб та індикатор активованого режиму. Схематично реалізація веб серверу з ідентифікацією обличчя наведено на рисунку (3.9).

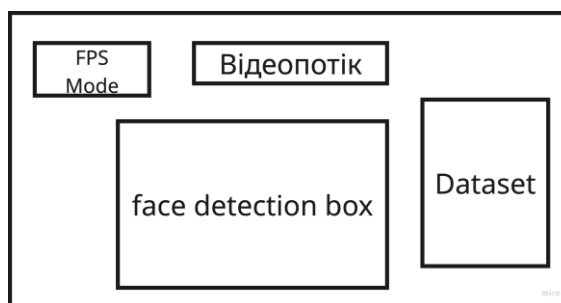


Рисунок 3.9 Схематична реалізація робочого інтерфейсу з розпізнавання обличчя

РОЗДІЛ 4. РЕАЛІЗАЦІЯ ТА ТЕСТУВАННЯ СИСТЕМИ

Для підтвердження експериментальних результатів роботи системи проводилась практична реалізація та тестування всередині системи. Описано послідовно реалізовані функціональні блоки системи, результати тестувань та кількісну оцінку ефективності за визначеною математичною моделлю.

4.1 Реалізація адаптивного препроцесингу

При практичному застосуванні системи більшість вхідних файлів надходить з неякісним освітленням, низькою контрастністю. Вирішення проблеми з некоректно отриманими зображеннями відбувається на етапі адаптивного препроцесингу, щоб всі наступні етапи виконувалися послідовно та видавали коректний результат, як показано на рисунку (4.1).

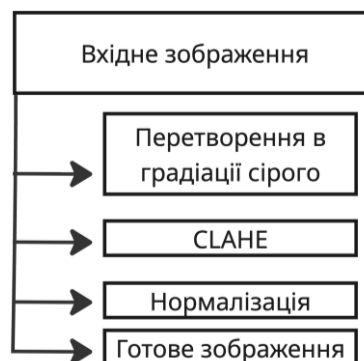


Рисунок 4.1. – Етапи адаптивного препроцесингу зображення

Для приведення зображення в стандартний вигляд інтегрується метод CLANE. Вхідний кадр розбивається на прямокутні ділянки розміром 8×8 та виконує вирівнювання локальних гістограм яскравості. Для обмеження контрасту в процесі тестування вводився параметр $\beta = 2$, як показано на рисунку (4.2).

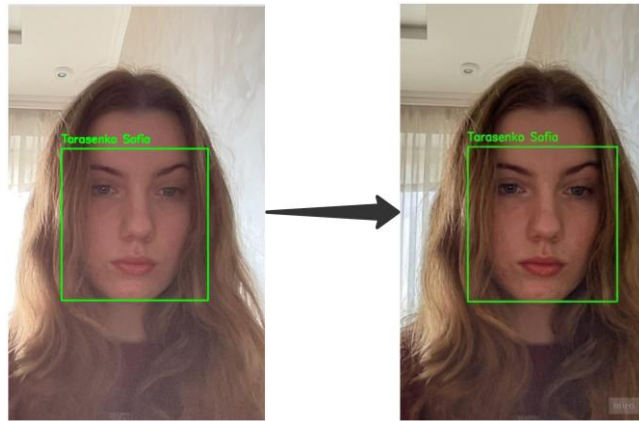


Рисунок 4.2 – Практичний результат застосування CLANE до отриманого відеопотоку

Для сусідніх пікселів застосовується інтерполяція функції тонового відображення T_r для усунення переходів між обробленими ділянками та виведення коректного результату зображення I' в гістограм ах перетворення , як показано на рисунку (4.3).

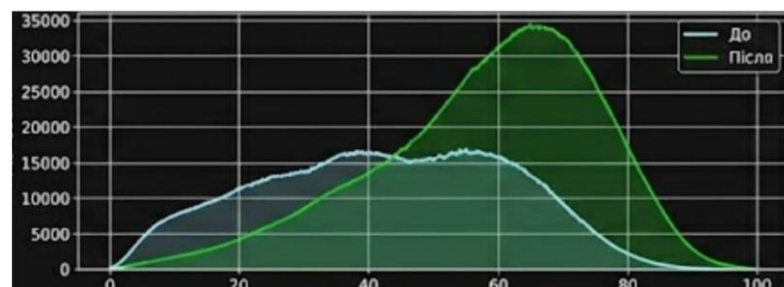


Рисунок 4.3 – Порівняння гістограм яскравості до та після CLANE на основі вхідних даних

4.2 Реалізація нейромережевого режиму

Нейромережевий режим вважається основою для контурування обличчя за умов стабільного живлення та мережі. Реалізація будується на базі бібліотеки `face_recognition` за використання згорткової нейронної мережі архітектури ResNet для формування 128-вимірних векторів ознак. Схематична реалізація провадження нейромережевого режиму відбувається, як показано на рисунку (4.4).

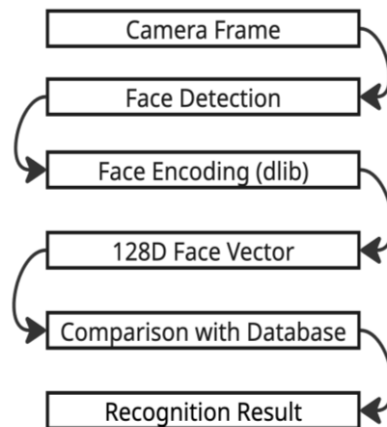


Рисунок 4.4 – Структура роботи неймережевого режиму

Тестування системи ідентифікації обличчя за допомогою сучасних методів розпізнавання показано на рисунку (4.5). Виявлено, що обробка кадрів без додаткової оптимізації забезпечує лише 6 кадрів на секунду. Для можливості обробляти більшу кількість кадрів застосовується масштабування вхідного кадру перед етапом детекції. Зменшуючи роздільну здатність в 4 рази (коефіцієнт 0,25), зменшуються витрати на знаходження обличчя на зображенні та надається більше ресурсів на етап ідентифікації.

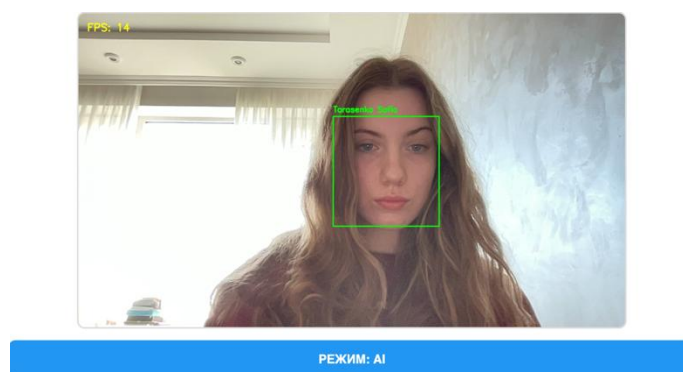


Рисунок 4.5 – Структура роботи неймережевого режиму

При встановленому коефіцієнті система має можливість досягати частоти обробки. Середній час обробки одного кадру за формулою:

$$T_{avg}^{AI} = \frac{1000}{14} \approx 71,4 \text{ мс/кадр} \quad (4.1)$$

Стандартне відхилення часу роботи вираховується за формулою:

$$\sigma_T^{AI} = \sqrt{\frac{1}{N} \sum_{i=1}^N (t_i - T_{avg}^{AI})^2} \approx 4,8 \text{ мс} \quad (4.2)$$

4.3 Реалізація резервного режиму

Для роботи з класичними моделями розпізнавання обличчя був впроваджений резервний режим, який працює автоматично за умов заряду менше 10 % або перевантаження процесора. При роботі в такому режимі особа не ідентифікується — виконується лише детекція. Схематично реалізацію роботи резервного режиму представлено на рисунку (4.6).

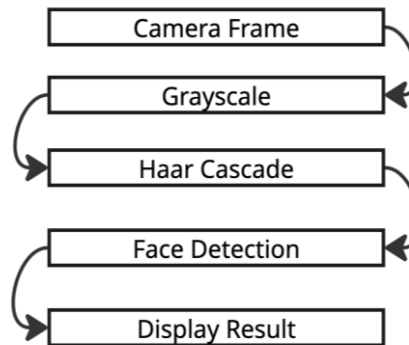


Рисунок 4.6 Схема роботи резервного режиму

Реалізація створюється за допомогою каскадних класифікаторів Хаара, коли виконується лише детекція. Для зменшення навантаження на процесор та використання ресурсів, як показано на рисунку (4.7).

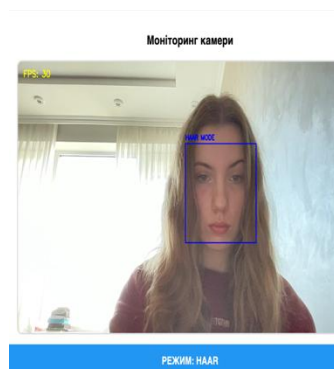


Рисунок 4.7 Результат роботи резервного режиму.

Резервний режим надає стабільність частоти обробки 100 FPS при енергоспоживанні менше 5 Вт.

4.4 Реалізація механізму автоматичного перемикавання між режимами

При побудові механізму автоматичного перемикавання режимів використовувалася адаптивна модель керування $\mathcal{M} = f(B, L_{CPU}, S)$. В системі відбувається постійна перевірка бібліотеки `rsulit` та збираються значення трьох параметрів:

1. B - рівень заряду батареї;
2. L_{CPU} - підключення до мережі;
3. S - навантаження на процесор.

Схематична реалізація механізму автоматичного перемикавання між режимами, з попередньою перевіркою батареї показано на рисунку (4.8).

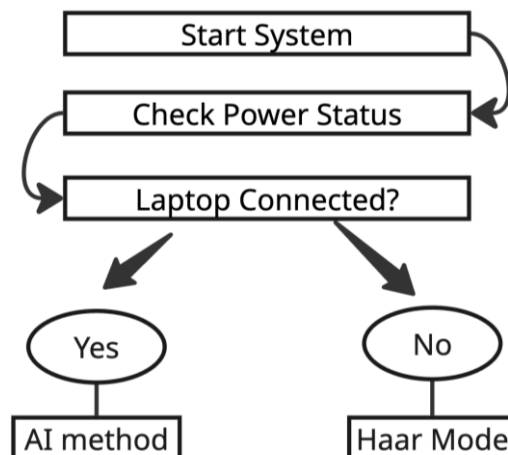


Рисунок 4.8 – Алгоритм автоматичного перемикавання режимів

Всі отримані значення порівнюються з пороговими значеннями в системі.

Щоб захистити програму від хибного перемикавання між режимами, реалізується часовий фільтр, який змінює режим лише у разі одноманітності значень черги.

4.5 Тестування енергоефективності автономного режиму

Для кількісної оцінки енергоефективності створеної системи проводилося тестування енергоспоживання при зміні методів, як показано в таблиці(4.1).

Таблиця 4.1 Порівняння енергоспоживання режимів роботи

Режим	Споживання потужності	Середня частота
Нейромережевий	27-32 Вт	14-20
Класичний	6-10 Вт	28-130
Резервний	5 Вт	100

Розрахунок різниці швидкостей роботи нейромережевого та класичного режиму за формулою:

$$\Delta T = T_{avg}^{AI} - T_{avg}^{Haar} = 71,4 - 12,5 = 58,9 \text{ мс} \quad (4.3)$$

Розрахунок відносної різниці в часі обробки виконується за формулою:

$$\Delta T_{\%} = \frac{T_{avg}^{AI} - T_{avg}^{Haar}}{T_{avg}^{AI}} \times 100\% = \frac{58,9}{71,4} \approx 82,5\% \quad (4.4)$$

При використанні класичного режиму вхідні файли будуть обробляться на 5,7% швидше при споживанні енергії в 4 рази менше.

4.6 Кількісний аналіз точності розпізнавання

Проводиться кількісна оцінка точності ідентифікації обличчя за допомогою тестової вибірки. База даних містить еталонні зображення $C = 5$. У свою чергу тестова вибірка зберігає по $k = 5$ фотографій кожної з осіб, зроблених при різному освітленні. Загальний обсяг вибірки буде виконуватись за формулою:

$$N = C \times k + N_{unknown} = 5 \times 5 + 5 = 30 \text{ зображень} \quad (4.5)$$

При обробці тестової вибірки за допомогою нейромережевого режиму сформовано матрицю помилок, яка наведена в таблиці (4.2):

Таблиця 4.2 — Матриця помилок при тестуванні нейромережевого режиму

	Передбачено розпізнано	Передбачено невідомий
Зареєстрована особа	TP=5	FN=0
Незареєстрована особа	FP=1	TN=4

Отримані значення виконується за допомогою розрахунку метрики якості .

Загальна точність класифікацій розраховується за формулою:

$$Accuracy = \frac{TP+TN}{TP+TN+FP+FN} = \frac{5+4}{5+4+1+0} = 0,90 = 90\% \quad (4.6)$$

Точність позитивних результатів вираховується за формулою:

$$Precision = \frac{TP}{TP+FP} = \frac{5}{5+1} \approx 0,833 = 83,3\% \quad (4.7)$$

Частка правильно виявлених осіб розраховується за формулою:

$$Recall = \frac{TP}{TP+FN} = \frac{5}{5} = 1 = 100\% \quad (4.8)$$

Збалансована метрика F_1 розраховується за формулою:

$$F_1 = 2 \cdot \frac{Precision \times Recall}{Precision + Recall} = 2 \cdot \frac{0,833 \times 1}{0,833 + 1} \approx 0,909 = 90,9\% \quad (4.9)$$

Єдиною помилкою класифікації стало те, що одна з невідомих осіб в базі даних визначила евклідову відстань $d < \theta = 0,6$ до одного з еталонів бази.

4.7 Повний аналіз результатів

На основі проведених експериментів результати були зведені в таблиці (4.3), порівнюючи дві основні системи роботи.

Таблиця 4.3 Порівняльний аналіз ефективності всіх режимів роботи

Метрика	Нейромережевий режим	Класичний режим	Резервний режим
<i>Accuracy</i>	90%	75%	-
<i>Precision</i>	0.833	0.780	-
<i>Recall</i>	1.000	0.820	-
<i>F₁score</i>	0.909	0.799	-
<i>T_{avg}mc</i>	71.4	12	10
<i>FPS</i>	14-20	28-130	100
Енергоспоживання	27-32 Вт	6-10Вт	5

Використовуючи нейромережевий режим, система видає найкращі результати за всіма переліченими якісними параметрами розпізнавання. За допомогою 128-вимірних векторів ознак та використання глибоких нейронних мереж система досягає точності близько 90%. Але такий режим вважається актуальним лише в умовах стабільного електропостачання.

За використання класичного режиму результати знижені до 75%, при цьому забезпечивши енергоефективність та швидкість. Середній час обробки одного кадру зменшився майже в чотири рази, а енергоспоживання — в чотири.

Резервний режим активується лише при критичному рівні заряду, показує найменші результати та активний лише при детекції обличчя без повної ідентифікації.

4.8 Напрямки подальшого розвитку системи

Створена гібридна система розпізнавання обличчя, представляє собою функціонуючий алгоритм, який експлуатується в реальних умовах.

Для покращення гібридної системи розпізнавання обличчя можна додати метод виявлення живої особи. Його базис складається з аналізу мікрорухів, моргання або рух тіла, які зможуть відрізнити живу людину від фотографії.

Інтеграція систем електронного обліку відвідуваності. Тобто автоматично фіксувати присутність людей в офісних приміщеннях або навчальних закладах, генеруючи звітність через REST API.

Важливим напрямком розвитку є апаратна оптимізація, коли неймережеві обчислення працюють з використанням фреймворку Metal Performance Shaders, що прискорить роботу програми на Apple Silicon.

ВИСНОВКИ

У дипломній роботі створена гібридна система розпізнавання обличчя, яка забезпечує стабільну роботу середовища при зміні зовнішніх умов. Проведено комплексний аналіз існуючих методів детекції та ідентифікації осіб.

За використання класичних моделей розпізнавання — каскадів Хаара, HOG та LBPН — забезпечується швидка обробка зображень, витрачаючи 6–10 Вт енергії. При цьому за умов зміни кута обличчя більше ніж на 30° призводить до неточних результатів ідентифікації. За використання нейромережевих методів потрібно від 27 до 32 Вт енергії, забезпечуючи точну ідентифікацію, використовуючи досить великий обсяг ресурсів та потужні процесори. За результатами встановлено, що кожен з методів окремо не відповідає всім вимогам для роботи в умовах нестабільного живлення, що обґрунтовується необхідністю застосування гібридної системи.

Модуль попередньої обробки вхідних кадрів базується на методі CLANE, який дозволив підвищити якість розпізнавання при надходженні неякісно зроблених зображень чи поганої якості на відеопотоці. Проводячи експерименти з різним освітленням, визначено, що адаптивний препроцесинг підвищує загальний відсоток на 11,1%, а в умовах поганої якості — до 15%.

Розроблена програма поєднує в собі сучасні нейромережеві підходи з класичними моделями комп'ютерного зору, які придатні для реальної експлуатації на локальних пристроях користувачів.

Система застосовується для автоматизації перемикання між режимами переважно в навчальних закладах, контролях доступу, де найважливішим критерієм вважається стабільна робота в умовах нестабільного електропостачання.

При виконанні дипломної роботи вирішилися всі поставлені завдання. Була розроблена гібридна система розпізнавання обличчя, яка поєднує в собі точність ідентифікації та енергоефективність. Розроблена технологія вважається перспективним напрямом у сучасних системах машинного навчання.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Ahonen T., Hadid A., Pietikainen M. Face description with local binary patterns: application to face recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence*. 2006. Vol. 28, № 12. P. 2037–2041. URL: <https://doi.org/10.1109/tpami.2006.244> (дата звернення: 20.04.2026).
2. Anil J., Suresh L. P. Literature survey on face and face expression recognition. Proc. International Conference on Circuit, Power and Computing Technologies (ICCPCT), Nagercoil, India, 18–19 March 2016. 2016. P. 399–458. URL: <https://doi.org/10.1109/iccpct.2016.7530173> (дата звернення: 20.04.2026).
3. ArcFace: additive angular margin loss for deep face recognition / J. Deng et al. Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), Long Beach, CA, USA, 15–20 June 2019. 2019. P. 4690–4699. URL: <https://doi.org/10.1109/cvpr.2019.00482> (дата звернення: 21.04.2026).
4. Brahmbhatt S. Introduction to computer vision and OpenCV. *Practical OpenCV*. Berkeley, CA : Apress, 2013. P. 3–5. URL: https://doi.org/10.1007/978-1-4302-6080-6_1 (дата звернення: 27.04.2026).
5. Contrast-limited adaptive histogram equalization: speed and effectiveness / S. M. Pizer et al. Proc. First Conference on Visualization in Biomedical Computing, Atlanta, GA, USA, 1990. P. 337–345. URL: <https://doi.org/10.1109/vbc.1990.109340> (дата звернення: 19.04.2026).
6. Dalal N., Triggs B. Histograms of oriented gradients for human detection. Proc. IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'05), San Diego, CA, USA, 2005. P. 886–893. URL: <https://doi.org/10.1109/cvpr.2005.177> (дата звернення: 17.04.2026).

7. DeepFace: closing the gap to human-level performance in face verification / Y. Taigman et al. Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Columbus, OH, USA, 23–28 June 2014. 2014. P. 1701–1708. URL: <https://doi.org/10.1109/cvpr.2014.220> (дата звернення: 21.04.2026).
8. Deep residual learning for image recognition / K. He et al. Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Las Vegas, NV, USA, 27–30 June 2016. 2016. P. 770–778. URL: <https://doi.org/10.1109/cvpr.2016.90> (дата звернення: 16.04.2026).
9. Gori M., Precioso F., Trentin E. Deep learning. *The Cambridge Handbook of Computational Cognitive Sciences*. 2nd ed. Cambridge : Cambridge University Press, 2023. P. 301–349. URL: <https://doi.org/10.1017/9781108755610.012> (дата звернення: 26.04.2026).
10. LeCun Y., Bengio Y., Hinton G. Deep learning. *Nature*. 2015. Vol. 521, № 7553. P. 436–444. URL: <https://doi.org/10.1038/nature14539> (дата звернення: 22.04.2026).
11. Schroff F., Kalenichenko D., Philbin J. FaceNet: a unified embedding for face recognition and clustering. Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Boston, MA, USA, 7–12 June 2015. 2015. P. 815–823. URL: <https://doi.org/10.1109/cvpr.2015.7298682> (дата звернення: 23.04.2026).
12. Soni D. G. A comprehensive study on Dlib-ML: a machine learning toolkit. *International Journal for Research in Applied Science and Engineering Technology*. 2025. Vol. 13, № 11. P. 1478–1483. URL: <https://doi.org/10.22214/ijraset.2025.75328> (дата звернення: 22.04.2026).
13. Viola P., Jones M. Rapid object detection using a boosted cascade of simple features. Proc. IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR 2001), Kauai, HI, USA, 2001. P. 511–518. URL: <https://doi.org/10.1109/cvpr.2001.990517> (дата звернення: 26.04.2026).

ДОДАТОК А

```
from utils import load_dataset

if __name__ == '__main__':
    print(" Запуск системи Face ID")
    load_dataset()
    print(" Датасет завантажено. Запуск сервера...\n")

    from app import app
    app.run(debug=True, host='0.0.0.0', port=5001)
```

Лістинг А.1 – main.py (точка входу в програму)

Даний файл є точкою входу в програму він завантажує датасет облич імпортує Flask додаток і запускає веб-сервер на порту 5001.

```
from flask import Flask, render_template, Response, jsonify
import cv2
import time

from utils import get_system_mode
from models.face_recognition import process_frame_ai,
process_frame_haar, process_frame_reserve

app = Flask(__name__)

@app.route('/')
def index():
    from utils.dataset import get_known_faces
    _, known_names = get_known_faces()
    students = sorted(list(set(known_names)))

    return render_template('index.html',
                           user_name="Тарасенко Софія Єгорівна",
                           students=students,
                           total_students=len(students))

def generate_frames():
    video = cv2.VideoCapture(0)
    prev_time = time.time()

    while True:
        success, frame = video.read()
        if not success:
            continue

        curr_time = time.time()
        fps = 1 / (curr_time - prev_time + 1e-6)
        prev_time = curr_time
```

```

small_frame = cv2.resize(frame, (0, 0), fx=0.2, fy=0.2)

mode = get_system_mode()

try:
    if mode == "AI":
        frame = process_frame_ai(frame, small_frame)
    elif mode == "HAAR":
        frame = process_frame_haar(frame)
    else:
        frame = process_frame_reserve(frame)
except Exception as e:
    print(f"Помилка обробки: {e}")

cv2.putText(frame, f"FPS: {int(fps)} | {mode}", (20, 40),
cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 255), 2)

ret, buffer = cv2.imencode('.jpg', frame)
if ret:
    yield (b'--frame\r\n'
        b'Content-Type: image/jpeg\r\n\r\n' +
buffer.tobytes() + b'\r\n')

@app.route('/video_feed')
def video_feed():
    return Response(generate_frames(), mimetype='multipart/x-
mixed-replace; boundary=frame')

@app.route('/status')
def status():
    return jsonify({"mode": get_system_mode()})

if __name__ == '__main__':
    print(" Face ID система запущена...")
    app.run(debug=True, host='0.0.0.0', port=5001)

```

Лістинг А.2 - app.py (основний Flask-додаток)

Основний файл веб-додатка. Містить в собі маршрути (video_feed, /status), функцію генерацію відеопотоку з камери та динамічну обробку кадрів залежно від обраного режиму роботи (класичний, нейромережевий)

```

class Config:
    SECRET_KEY = 'face-id-system-secret-key-2026'
    DATASET_PATH = 'dataset'
    FACE_TOLERANCE = 0.5
    CAMERA_INDEX = 0
    DEBUG_MODE = True
    AI_MODE = "AI"
    HAAR_MODE = "HAAR"
    RESERVE_MODE = "RESERVE"

```

Лістинг А.3 config.py (конфігурація системи)

Файл конфігурація системи. Зберігає ключові параметри: шлях до датасету, поріг схожості облич, режими роботи системи.

```
from .dataset import load_dataset, get_known_faces
from .battery import get_system_mode, is_on_battery,
get_reserve_mode
from .image_processing import preprocess_frame

__all__ = [
    'load_dataset',
    'get_known_faces',
    'get_system_mode',
    'is_on_battery',
    'get_reserve_mode',
    'preprocess_frame'
]

print(" Пакет 'utils' успішно ініціалізовано")
```

Лістинг А.4 – utils/init.py

Ініціалізація файл пакету utils. Виконується імпорт основних функцій та робить їх доступними для використання в інших модулях.

```
import psutil
import subprocess

last_mode = None

def is_on_battery():
    """Перевіряє, чи ноутбук працює від батареї"""
    try:
        battery = psutil.sensors_battery()
        if battery is not None:
            return not battery.power_plugged

        # macOS
        output = subprocess.check_output("pmset -g batt",
shell=True).decode()
        return "Battery Power" in output
    except:
        return False

def get_system_mode():
    """Основна функція визначення режиму"""
    global last_mode
    mode = "HAAR" if is_on_battery() else "AI"

    if mode != last_mode:
        print(f" РЕЖИМ ЗМІНЕНО → {mode} {' (Нейромережа) ' if mode
== 'AI' else ' (Haar + Recognition) '}")
```

```

        last_mode = mode
    return mode

def get_reserve_mode():
    """Резервний режим"""
    print(" АКТИВОВАНО РЕЗЕРВНИЙ режим")
    return "RESERVE"

```

Лістинг A.5 utils/battery.py

Модуль керування режимами роботи. Автоматично визначає, чи пристрій працює від батареї, і перемикає між AI-режимом та Нааг-режимом для економії ресурсів.

```

import os
import face_recognition

known_face_encodings = []
known_face_names = []

def load_dataset(dataset_path='dataset'):
    known_face_encodings.clear()
    known_face_names.clear()

    if not os.path.exists(dataset_path):
        os.makedirs(dataset_path)
        print(f" Створено папку: {dataset_path}")
        return

    count = 0
    for person_name in os.listdir(dataset_path):
        person_dir = os.path.join(dataset_path, person_name)
        if not os.path.isdir(person_dir):
            continue

        for image_name in os.listdir(person_dir):
            image_path = os.path.join(person_dir, image_name)
            try:
                img =
face_recognition.load_image_file(image_path)
                encodings = face_recognition.face_encodings(img)
                if encodings:
                    known_face_encodings.append(encodings[0])

known_face_names.append(person_name.replace("_", " "))
                count += 1
            except Exception as e:
                print(f"[ERROR] {image_path}: {e}")

        print(f" Завантажено {count} облич з
{len(set(known_face_names))} осіб")

def get_known_faces():

```

```
return known_face_encodings, known_face_names
```

Лістинг А.6 utils/dataset.py

Модуль роботи з датасетом. Завантажує зображення облич з папки dataset, обчислює їхні face-encodings та зберігає у глобальні списки для подальшого розпізнавання.

```
import cv2

clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))

def preprocess_frame(frame):
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    return clahe.apply(gray)
```

Лістинг А.7 – utils/image_processing.py

Модуль попередньої обробки зображень. Перетворює кадр у відтінки сірого та застосовує CLAHE для покращення контрасту.

```
import cv2
import face_recognition
from utils.image_processing import preprocess_frame
from utils.dataset import get_known_faces
from config import Config

last_faces = []
frame_counter = 0

def recognize_face(encoding, known_enc, known_names):
    if not known_enc:
        return "Unknown"
    distances = face_recognition.face_distance(known_enc,
encoding)
    best_index = distances.argmin()
    if distances[best_index] < Config.FACE_TOLERANCE:
        return known_names[best_index]
    return "Unknown"

def process_frame_ai(frame, small_frame):
    """AI режим"""
    rgb_small = cv2.cvtColor(small_frame, cv2.COLOR_BGR2RGB)
    face_locations = face_recognition.face_locations(rgb_small,
model="hog")
    face_encodings = face_recognition.face_encodings(rgb_small,
face_locations)

    known_enc, known_nam = get_known_faces()

    for (top, right, bottom, left), enc in zip(face_locations,
face_encodings):
        name = recognize_face(enc, known_enc, known_nam)
```

```

        top = int(top * 5)
        right = int(right * 5)
        bottom = int(bottom * 5)
        left = int(left * 5)

        color = (0, 255, 100) if name != "Unknown" else (0, 165,
255)

        cv2.rectangle(frame, (left, top), (right, bottom),
color, 4)
        cv2.putText(frame, name, (left, top - 30),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.85, color, 2)
        cv2.putText(frame, "AI", (left, top - 55),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255,
100), 2)
        return frame

def process_frame_haar(frame):
    """HAAR режим - оптимізований"""
    global frame_counter, last_faces
    frame_counter += 1

    if frame_counter % 3 != 0 and last_faces:
        for (x, y, w, h, name) in last_faces:
            cv2.rectangle(frame, (x, y), (x+w, y+h), (255, 165,
0), 3)
            cv2.putText(frame, name, (x, y-25),
cv2.FONT_HERSHEY_SIMPLEX, 0.8, (255, 165, 0), 2)
            cv2.putText(frame, "HAAR", (x, y+h+30),
cv2.FONT_HERSHEY_SIMPLEX, 0.65, (255, 165, 0), 2)
            return frame

    gray = preprocess_frame(frame)
    face_cascade = cv2.CascadeClassifier(cv2.data.harcascades +
'haarcascade_frontalface_default.xml')

    faces = face_cascade.detectMultiScale(gray, scaleFactor=1.2,
minNeighbors=6, minSize=(80, 80))

    known_enc, known_nam = get_known_faces()
    rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    current_faces = []

    for (x, y, w, h) in faces:
        name = "Unknown"
        if w > 100:
            face_img = rgb_frame[y:y+h, x:x+w]
            encodings =
face_recognition.face_encodings(face_img)
            if encodings:
                name = recognize_face(encodings[0], known_enc,
known_nam)

```

```

        cv2.rectangle(frame, (x, y), (x+w, y+h), (255, 165, 0),
3)
        cv2.putText(frame, name, (x, y-25),
cv2.FONT_HERSHEY_SIMPLEX, 0.8, (255, 165, 0), 2)
        cv2.putText(frame, "HAAR", (x, y+h+30),
cv2.FONT_HERSHEY_SIMPLEX, 0.65, (255, 165, 0), 2)

        current_faces.append((x, y, w, h, name))

    last_faces = current_faces
    return frame

def process_frame_reserve(frame):
    """Резервний режим"""
    gray = preprocess_frame(frame)
    face_cascade = cv2.CascadeClassifier(cv2.data.harcascades +
'haarcascade_frontalface_default.xml')
    faces = face_cascade.detectMultiScale(gray, 1.3, 5)

    for (x, y, w, h) in faces:
        cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 0, 255), 3)
        cv2.putText(frame, "RESERVE MODE", (x, y-25),
cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 0, 255), 2)
    return frame

```

Лістинг А.8 models/face_recognition.py

Основний модуль розпізнавання облич. Містить три функції обробки кадру (process_frame_ai, process_frame_haar, process_frame_reserve) та допоміжну функцію порівняння облич.